LLM Security PoC — Pipeline Runbook

Automated 90-sample evaluation of security vulnerabilities in LLM-generated web application code

Repository: **llm-security-poc** · University of Moratuwa MSc · generated 13 Oct 2026

This runbook explains what is in the repository, which parts of the proposal it implements, exactly how to run it end to end, and the few things you must provide. It is written for a GitHub Codespaces environment, which is where the heavy tools (Docker, Semgrep, CodeQL, ZAP) run.

1. What's in the repository

The pipeline turns one configuration into all 90 samples — generated, extracted, made runnable, statically analysed, attacked, and scored. Everything lives under `scripts/` and `prompts/`; results land in four output folders.

Configuration & prompts

File	Purpose
scripts/poc_config.py	Single source of truth: the matrix (5 tasks × 2 models × 3 strategies × 3 reps = 90), model settings, repo paths. Widen here to scale.
prompts/_contract.txt	The output contract appended to every prompt — forces a JSON file manifest. This is what fixed extraction and the missing index.html / package.json.
prompts/tasks/*.txt	The 5 benchmark task specs: a01, a03, a07, a10, cross (functional requirements).
prompts/strategies/*.txt	The 3 prompting strategies: zero_shot, security_instructed, few_shot.

Pipeline scripts

Script	Phase	What it does
generate_code.py	1	Calls the model(s) per the matrix; saves raw response to generated-code/<sample>.md.
lib_extract.py / extract_all.py	1	Parse response into project files — JSON manifest first, markdown fallback.
lib_scaffold.py / scaffold_all.py	1	Inject missing boilerplate (index.html, entry, package.json, .env), log it, and draft contract.json.
run_semgrep.py	3	Semgrep static analysis → CWE set.
run_codeql.py	3	CodeQL static analysis (SARIF → CWE); skips cleanly if the CLI is absent.
llm_judge.py	3	LLM security judge, CWE-tagged, with false-positive flagging.
phase3_static.py	3	Runs the three layers, writes master_results.csv + evaluator_matrix.csv.
attacks.py	5	Per-task runtime attacks (a01/a03/a07/a10/cross), contract-driven, cookie-aware.
run_zap.py	5	OWASP ZAP dynamic scan.

Script	Phase	What it does
phase6_metrics.py	6	ASR, Confirmation Rate, Attack Diversity H, Secure / SecurePass; fills the matrix.
run_all.py	1–3	Orchestrator: runs the cheap phases across the matrix. Resumable, parallel, filterable.
run_sample.sh	4–6	Serial per-sample driver: deploy → ZAP + attacks → metrics → teardown.
setup_pipeline.sh	setup	Installs local Semgrep rules + the CodeQL bundle.

Output folders: `generated-code/` (raw + extracted apps), `static-analysis/` (Semgrep/CodeQL/judge), `red-teaming/` (attacks/ZAP), `results/` (the two CSVs — the single source of truth).

2. What it implements from the proposal

Proposal element	Where in the pipeline
Phase 1 — LLM code collection (§3.2.3): 5 tasks × 2 models × 3 strategies × 3 reps = 90	poc_config.py + generate_code.py + extract/scaffold
OWASP Top 10 benchmark tasks (Table 3.1): A01, A03, A07, A10, cross-cutting	prompts/tasks/*.txt
Phase 2 — Functional correctness / Pass@1	build check inside run_sample.sh
Phase 3 — Three-layer static analysis (§3.3.1): Semgrep + CodeQL + LLM judge	run_semgrep.py, run_codeql.py, llm_judge.py, phase3_static.py
Phase 3/5 — Evaluator complementarity → Cohen's / Fleiss' κ (§3.6.4) and MI / entropy (§3.6.5)	results/evaluator_matrix.csv (one row per sample × CWE)
Phase 5 — Adaptive red teaming (§3.4): per-OWASP attacks + DAST	attacks.py (per task) + run_zap.py
Phase 6 — Metrics (§3.6.6): ASR, Confirmation Rate, Attack Diversity H (Eq 3.5)	phase6_metrics.py
Security metrics (§3.6.2): Secure@k, SecurePass@k, vulnerability density	phase3_static.py + phase6_metrics.py
PoC success criteria C1–C6 (Table 3.4)	columns across master_results.csv
Minimal adaptive memory loop for C6 (guide §9)	attacks.py --adaptive

3. What you need to provide

- **An OpenAI API key** added as a Codespaces secret named `OPENAI_API_KEY`. (DeepSeek comes later as `DEEPSEEK_API_KEY` — not needed for the first 45 samples.)
- **Run in GitHub Codespaces**, not on your Mac — the Mac has no Docker, Semgrep or CodeQL. Codespaces has them.
- **Review** `prompts/tasks/*.txt` once before generating — these four new task specs (a01, a03, a10, cross) were drafted from proposal Table 3.1 and define your benchmark. a07 is your original text.
- **Skim each** `contract.json` (marked `_needs_review`) before running Phase 5 on an app — confirm the drafted routes/field names match that app.
- **Commit to git** before the first run, so your existing hand-made samples are preserved.

Nothing else in the code needs editing — the scripts read the keys from the environment and the matrix from poc_config.py.

4. Step-by-step: running it (GPT-4o, 45 samples)

Step 0 — Add your key & open Codespaces

At `github.com/settings/codespaces` → Secrets → New secret: name `OPENAI_API_KEY`, give it access to the `llm-security-poc` repo. Then open the repo in a Codespace (secrets load at container start, so add the secret first, or rebuild the container afterwards).

Step 1 — Checkpoint in git

```
cd llm-security-poc
git add -A && git commit -m "Add automated 90-sample pipeline"
```

Step 2 — One-time setup (tools + rules)

```
bash scripts/setup_pipeline.sh
export SEMGREP_RULES="$PWD/.tooling/semgrep-rules/javascript"
export CODEQL="$PWD/.tooling/codeql/codeql"
[ -n "$OPENAI_API_KEY" ] && echo "key ok"
```

Step 3 — Smoke-test ONE sample first (~$0.10)

Confirms generation, extraction, scaffolding and static analysis all work before you spend on 45.

```
python3 scripts/run_all.py --only a01_gpt4o_zero_shot_rep1
```

Check that `generated-code/a01_gpt4o_zero_shot_rep1_files/` contains a `public/index.html`, a `contract.json`, and that a row appeared in `results/master_results.csv`.

Step 4 — Run all 45 GPT-4o samples, Phases 1–3

```
python3 scripts/run_all.py --models gpt4o --stages generate,extract,scaffold,static
```

Resumable — if it stops, run the same line again and it continues. Static analysis (especially CodeQL) is the slow part; budget a few hours. Approx API cost: $3–6.

Step 5 — Review the results

Open `results/master_results.csv` (one row per sample) and `results/evaluator_matrix.csv` (one row per sample × CWE).

Step 6 — Phases 4–6, one sample at a time (runtime attacks)

First open that app's `contract.json` and confirm the routes. Then:

```
sudo iptables -P FORWARD ACCEPT     # one-time, Codespaces Docker fix
bash scripts/run_sample.sh a07_gpt4o_zero_shot_rep3
```

This deploys the app (Postgres in Docker, backend native), runs ZAP + the per-task attacks, writes the runtime metrics, and tears down. Works for all five task types.

Step 7 — Later: add DeepSeek (the other 45)

Add the `DEEPSEEK_API_KEY` secret, rebuild the Codespace, then one command fills in the remaining rows. Your GPT-4o results are untouched (resumable state).

```
python3 scripts/run_all.py --models deepseek --stages generate,extract,scaffold,static
```

5. Outputs and how they feed the analysis

Column (master_results.csv)	Proposal metric
functional_pass → Pass@1	Functional correctness (Eq 3.1)
static_cwe_set, static_count, vuln_density_kloc	Static findings & vulnerability density (§3.6.2)
asr_pct	Attack Success Rate per OWASP category (§3.6.6)
confirmation_rate, runtime_only_count	Static-vs-dynamic confirmation (Experiments 3–4)
attack_diversity_H	Attack Diversity Index H (Eq 3.5)
secure, securepass	Secure@k / SecurePass@k (Eq 3.2)
evaluator_matrix.csv (semgrep/codeql/llm_judge/runtime_confirmed)	Cohen's/Fleiss' κ (§3.6.4), MI/entropy (§3.6.5)

6. Notes & good practice

- **Verified in build:** extraction + scaffolding on all 9 existing A07 samples; Semgrep producing the expected CWEs; a full live Phase 4–6 run on rep3; and all five attack suites tested against vulnerable and secure mock apps (fire correctly, zero false positives on the secure ones).
- **contract.json is a best-effort draft** (marked _needs_review). If a route is wrong, that one attack skips safely rather than mis-scoring — but a quick review per app is worth it.
- **Generate one sample per task first** (e.g. a01/a03/a07/a10/cross, zero_shot, rep1), inspect, then run the full set.
- **C6 (adaptive loop):** add `--adaptive` to attacks for the minimal memory loop, or run `run_sample.sh` which already passes it.
- **Research integrity:** the scaffolder never edits model-written code — it only adds boilerplate and logs every addition, so your vulnerability findings stay valid.

Full reference: `PIPELINE_README.md` in the repo. Phase infrastructure details and risk register: `EXECUTION_GUIDE.md`.